

Machine Learning Practice Problems

Prof. Saurabh

February 14, 2026

Contents

1	Introduction and Machine Learning Workflow	2
2	Linear Models for Regression	7
3	Linear Models for Classification	11
4	Decision Tree Learning	15
5	Instance-based Learning	19
6	Support Vector Machines (SVM)	23
7	Bayesian Learning	27
8	Ensemble Learning	32
9	Unsupervised Learning	35
10	Model Evaluation	39
11	Emerging Requirements in ML	43

1 Introduction and Machine Learning Workflow

Question 1.1: Conceptual

A credit card company wants to detect fraudulent transactions. They have a dataset of past transactions labeled as 'fraud' or 'legitimate'. What type of machine learning problem is this? If they had no labels and just wanted to group similar transactions together to find anomalies, what type would it be?

Solution

This problem involves two distinct scenarios:

1. **Scenario 1: Labeled Data ('fraud' or 'legitimate')**
 - **Type:** Supervised Learning.
 - **Task:** Classification.
 - **Reasoning:** The goal is to predict a discrete label (class) based on past examples where the correct answer (label) is known.
2. **Scenario 2: No Labels**
 - **Type:** Unsupervised Learning.
 - **Task:** Clustering or Anomaly Detection.
 - **Reasoning:** Without predefined labels, the goal is to discover hidden structures or groups in the data. Grouping similar transactions is *Clustering*, while finding transactions that deviate significantly from the norm is *Anomaly Detection*.

Question 1.2: Conceptual

Given a feature x with values $\{10, 20, 30, 40, 50\}$. Perform Min-Max Normalization to scale these values to the range $[0, 1]$. What is the normalized value of 30?

Solution

Min-Max Normalization Formula:

$$x_{norm} = \frac{x - x_{min}}{x_{max} - x_{min}}$$

Step-by-Step Calculation:

1. Identify the minimum and maximum values in the dataset:

$$x_{min} = 10, \quad x_{max} = 50$$

2. Substitute the values for $x = 30$:

$$x_{norm} = \frac{30 - 10}{50 - 10}$$

3. Simplify the numerator and denominator:

$$x_{norm} = \frac{20}{40}$$

4. Calculate the final result:

$$x_{norm} = 0.5$$

The normalized value of 30 is **0.5**.

Question 1.3: Numerical

Given a dataset with mean $\mu = 50$ and standard deviation $\sigma = 5$. Calculate the Z-score standardization for a data point with value $x = 65$. What does this Z-score indicate?

Solution

Z-score Formula:

$$z = \frac{x - \mu}{\sigma}$$

Step-by-Step Calculation:

1. Substitute the given values ($x = 65, \mu = 50, \sigma = 5$):

$$z = \frac{65 - 50}{5}$$

2. Simplify the numerator:

$$z = \frac{15}{5}$$

3. Calculate the final result:

$$z = 3$$

Interpretation: A Z-score of 3 indicates that the value 65 is **3 standard deviations above the mean**.

Question 1.4: Conceptual

Explain the 'Curse of Dimensionality' in the context of distance-based algorithms like K-Nearest Neighbors (KNN). Why does increasing the number of features potentially degrade performance?

Solution

The **Curse of Dimensionality** refers to the phenomena that arise when analyzing data in high-dimensional spaces.

Key Issues for Distance-Based Algorithms (like KNN):

- **Data Sparsity:** As the number of dimensions (features) increases, the volume of the space increases exponentially. To maintain the same data density, the amount of data required grows exponentially. With a fixed amount of data, the space becomes extremely sparse.
- **Distance Concentration:** In high-dimensional spaces, the distance between any two points tends to become equidistant. The ratio of the distance to the nearest neighbor to the distance to the farthest neighbor approaches 1.

$$\lim_{d \rightarrow \infty} \frac{\text{dist}_{\max} - \text{dist}_{\min}}{\text{dist}_{\min}} \rightarrow 0$$

- **Degraded Performance:** Because all points appear roughly equally far apart, the concept of "nearest neighbor" becomes meaningless. The algorithm struggles to distinguish between relevant (similar) and irrelevant (dissimilar) points, leading to poor classification or regression performance.

Question 1.5: Numerical

In a Principal Component Analysis (PCA), the eigenvalues of the covariance matrix are $\lambda_1 = 4.0$, $\lambda_2 = 2.0$, and $\lambda_3 = 0.5$. Calculate the proportion of total variance explained by the first two principal components.

Solution

Step-by-Step Calculation:

1. **Calculate Total Variance:** The total variance is the sum of all eigenvalues.

$$\lambda_{total} = \lambda_1 + \lambda_2 + \lambda_3 = 4.0 + 2.0 + 0.5 = 6.5$$

2. **Calculate Variance Explained by First Two Components:** Sum the eigenvalues corresponding to PC1 and PC2.

$$\lambda_{PC1+PC2} = \lambda_1 + \lambda_2 = 4.0 + 2.0 = 6.0$$

3. **Calculate Proportion:** Divide the explained variance by the total variance.

$$\text{Proportion} = \frac{\lambda_{PC1+PC2}}{\lambda_{total}} = \frac{6.0}{6.5}$$

4. **Convert to Percentage:**

$$\frac{6.0}{6.5} \approx 0.923 \quad \text{or} \quad 92.3\%$$

The first two principal components explain approximately **92.3%** of the total variance.

Question 1.6: Conceptual

Why is it necessary to split data into Training, Validation, and Test sets? Why can't we just use Training and Test sets?

Solution

Splitting data into three sets is crucial for building robust models and avoiding bias.

- **Training Set:** Used to fit the model parameters (weights, biases).
- **Validation Set:** Used to tune hyperparameters (e.g., learning rate, tree depth, regularization strength) and for model selection. It acts as a proxy for unseen data during development.
- **Test Set:** Used strictly for the **final evaluation** of the selected model to estimate its generalization error. It should never be used during training or tuning.

Why not just Train/Test? If we use the Test set for hyperparameter tuning, we are effectively "peeking" at the answers. We would select hyperparameters that happen to work well on that specific test set, leading to **data leakage** and an optimistic bias. The final performance estimate would be unreliable because the model has indirectly "seen" the test data. Using a separate Validation set prevents this overfitting to the test data.

Question 1.7: Conceptual

Describe two techniques to handle an imbalanced dataset where the positive class is very rare (e.g., 1% of data).

Solution

Two common techniques are:

1. Resampling Techniques:

- **Undersampling:** Randomly remove examples from the majority class to balance the class distribution. This is simple but risks discarding useful information.
- **Oversampling:** Randomly duplicate examples from the minority class or generate synthetic examples (e.g., using **SMOTE** - Synthetic Minority Over-sampling Technique) to increase the representation of the minority class.

2. Cost-sensitive Learning:

Modify the learning algorithm to penalize misclassifications of the minority class more heavily than the majority class. This is often done by assigning higher weights to minority class samples in the loss function.

$$\text{Loss} = w_{\text{minority}} \cdot \text{Loss}_{\text{minority}} + w_{\text{majority}} \cdot \text{Loss}_{\text{majority}}$$

where $w_{\text{minority}} > w_{\text{majority}}$.

Question 1.8: Conceptual

Given the feature values $F = \{2, 4, \text{NaN}, 8, 10, \text{NaN}\}$. a) Impute the missing values using the Mean. b) Impute the missing values using the Median.

Solution

Observed Values: $\{2, 4, 8, 10\}$ (ignoring NaNs).

a) Mean Imputation:

1. Calculate the mean of observed values:

$$\mu = \frac{2 + 4 + 8 + 10}{4} = \frac{24}{4} = 6$$

2. Replace NaNs with 6.

3. **Result:** $\{2, 4, 6, 8, 10, 6\}$

b) Median Imputation:

1. Sort the observed values: $\{2, 4, 8, 10\}$.
2. Calculate the median. Since there is an even number of values (4), take the average of the two middle values (4 and 8):

$$\text{Median} = \frac{4 + 8}{2} = 6$$

3. Replace NaNs with 6.

4. **Result:** $\{2, 4, 6, 8, 10, 6\}$

Question 1.9: Conceptual

You train a decision tree and observe the following: Training Accuracy = 99%, Validation Accuracy = 65%. What problem is the model suffering from? Suggest two ways to fix it.

Solution

Diagnosis: The large gap between the high Training Accuracy (99%) and the low Validation Accuracy (65%) indicates **Overfitting** (High Variance). The model has memorized the noise and specific details of the training data rather than learning the underlying general patterns.

Fixes:

1. **Pruning:** Reduce the complexity of the tree.
 - **Pre-pruning:** Set constraints like maximum depth (' max_depth '), minimum samples per split (' min_samples_split '), or minimum samples per leaf (' min_samples_leaf ').
 - **Post-pruning:** Grow the tree fully and then remove branches that provide little predictive power (e.g., Cost Complexity Pruning).
2. **More Data:** Increasing the size of the training dataset can help the model generalize better and reduce variance.

Question 1.10: Conceptual

You are using a K-Nearest Neighbors (KNN) algorithm and a Decision Tree algorithm on a dataset with features having very different scales (e.g., Age in years vs Income in dollars). For which algorithm is feature scaling more critical and why?

Solution

Feature scaling is **critical for KNN** but generally **not required for Decision Trees**.
Reasoning:

- **KNN (Critical):** KNN relies on calculating distances (e.g., Euclidean distance) between data points.

$$d(p, q) = \sqrt{\sum (p_i - q_i)^2}$$

If one feature has a large range (e.g., Income: 0-100,000) and another has a small range (e.g., Age: 0-100), the distance calculation will be completely dominated by the feature with the larger range. Income differences would overshadow Age differences. Scaling ensures all features contribute equally to the distance metric.

- **Decision Trees (Not Critical):** Decision Trees make splits based on single features at a time (univariate splits). They look for a threshold t for feature x_i such that $x_i < t$ vs $x_i \geq t$ maximizes information gain. This process depends only on the ordering of the values, not their magnitude. Monotonic transformations (like scaling) do not change the optimal split point or the tree structure.

2 Linear Models for Regression

Question 2.1: Conceptual

Given three data points (x, y) : $(1, 2), (2, 3), (3, 5)$. Use the method of least squares to find the best fit line $y = w_0 + w_1x$. (You can use the formula $w_1 = \frac{\sum(x_i - \bar{x})(y_i - \bar{y})}{\sum(x_i - \bar{x})^2}$ and $w_0 = \bar{y} - w_1\bar{x}$).

Solution

Step 1: Calculate Means

$$\bar{x} = \frac{1 + 2 + 3}{3} = \frac{6}{3} = 2$$

$$\bar{y} = \frac{2 + 3 + 5}{3} = \frac{10}{3} \approx 3.33$$

Step 2: Calculate Slope (w_1) Numerator: $\sum(x_i - \bar{x})(y_i - \bar{y})$

$$(1 - 2)(2 - 3.33) = (-1)(-1.33) = 1.33$$

$$(2 - 2)(3 - 3.33) = (0)(-0.33) = 0$$

$$(3 - 2)(5 - 3.33) = (1)(1.67) = 1.67$$

$$\text{Sum} = 1.33 + 0 + 1.67 = 3.0$$

Denominator: $\sum(x_i - \bar{x})^2$

$$(1 - 2)^2 = (-1)^2 = 1$$

$$(2 - 2)^2 = (0)^2 = 0$$

$$(3 - 2)^2 = (1)^2 = 1$$

$$\text{Sum} = 1 + 0 + 1 = 2$$

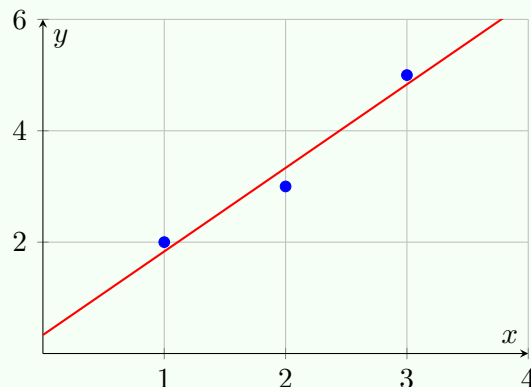
$$w_1 = \frac{3.0}{2} = 1.5$$

Step 3: Calculate Intercept (w_0)

$$w_0 = \bar{y} - w_1\bar{x} = 3.33 - 1.5(2) = 3.33 - 3.0 = 0.33$$

Final Equation:

$$y = 0.33 + 1.5x$$



Question 2.2: Numerical

Consider a simple regression model $y = wx$ (intercept is 0). You have one data point ($x = 2, y = 4$). Initial weight $w = 1$. Learning rate $\eta = 0.1$. Loss function $J(w) = \frac{1}{2}(y_{pred} - y)^2$. Perform one step of Gradient Descent to update w .

Solution

Step 1: Prediction

$$y_{pred} = w \cdot x = 1 \cdot 2 = 2$$

Step 2: Error

$$\text{Error} = y_{pred} - y = 2 - 4 = -2$$

Step 3: Gradient Calculation

The gradient of the loss function with respect to w is:

$$\frac{\partial J}{\partial w} = (y_{pred} - y) \cdot x$$

$$\text{Gradient} = (-2) \cdot 2 = -4$$

Step 4: Update Rule

$$w_{new} = w_{old} - \eta \cdot \text{Gradient}$$

$$w_{new} = 1 - 0.1(-4)$$

$$w_{new} = 1 + 0.4 = 1.4$$

The updated weight is **1.4**.

Question 2.3: Conceptual

Compare L1 (Lasso) and L2 (Ridge) regularization. Which one is preferred if you suspect many features are irrelevant and you want a sparse model?

Solution

- **L1 Regularization (Lasso):** Adds a penalty proportional to the absolute value of the weights ($\lambda \sum |w_i|$).
 - **Effect:** Shrinks some coefficients exactly to zero.
 - **Result:** Performs feature selection, yielding sparse models.
- **L2 Regularization (Ridge):** Adds a penalty proportional to the square of the weights ($\lambda \sum w_i^2$).
 - **Effect:** Shrinks coefficients towards zero but rarely makes them exactly zero.
 - **Result:** Handles multicollinearity well but keeps all features.

Preferred Choice: If you suspect many features are irrelevant, **L1 (Lasso)** is preferred because it will set the weights of those irrelevant features to zero, effectively removing them from the model.

Question 2.4: Numerical

Calculate the Ridge Regression cost function $J(w)$ for a model with weights $w = [3, 4]$, data error (MSE) = 10, and regularization parameter $\lambda = 0.5$. (Cost formula: $J = \text{MSE} + \lambda \sum w_i^2$).

Solution

Step 1: Calculate Regularization Term

$$\sum w_i^2 = 3^2 + 4^2 = 9 + 16 = 25$$

$$\text{Penalty} = \lambda \sum w_i^2 = 0.5 \times 25 = 12.5$$

Step 2: Calculate Total Cost

$$J = \text{MSE} + \text{Penalty} = 10 + 12.5 = 22.5$$

Question 2.5: Conceptual

You fit a polynomial regression model of degree 1 (linear), degree 5, and degree 20 to a dataset with 50 points. Which model is most likely to suffer from High Bias? Which is most likely to suffer from High Variance?

Solution

- **Degree 1 (Linear):** High Bias (Underfitting). It is too simple to capture complex non-linear patterns.
- **Degree 20:** High Variance (Overfitting). With 20 degrees of freedom for only 50 points, it will likely oscillate wildly to fit the noise in the data.

Question 2.6: Numerical

Explain the difference between Batch Gradient Descent and Stochastic Gradient Descent (SGD) in terms of parameter updates and convergence speed.

Solution

Feature	Batch Gradient Descent	Stochastic Gradient Descent (SGD)
Update Frequency	Once per epoch (after processing all N training examples).	N times per epoch (after every single training example).
Gradient Calculation	Exact gradient of the entire cost function.	Approximation of the gradient using a single example.
Convergence	Stable, straight path to the minimum. Slow for large datasets.	Noisy, oscillating path. Can escape shallow local minima. Much faster per iteration.

Question 2.7: Numerical

A regression model predicts values such that the Sum of Squared Residuals (SSR) is 20. The Total Sum of Squares (SST) of the target variable is 100. Calculate the R^2 score.

Solution

R^2 Score Formula:

$$R^2 = 1 - \frac{\text{SSR}}{\text{SST}}$$
$$R^2 = 1 - \frac{20}{100} = 1 - 0.2 = 0.8$$

The model explains **80%** of the variance in the target variable.

Question 2.8: Numerical

What happens if the learning rate η in Gradient Descent is too large? What happens if it is too small?

Solution

- **Too Large:** The algorithm might overshoot the minimum. This can cause the loss to oscillate or even diverge (increase) instead of converging.
- **Too Small:** The algorithm will take tiny steps. Convergence will be extremely slow, wasting computational resources, and it is more likely to get stuck in local minima.

Question 2.9: Conceptual

You are trying to predict the price of a house based on its size. You suspect the relationship is quadratic (price increases with size but levels off). How can you use linear regression to model this?

Solution

Polynomial Regression (Basis Expansion): Even though the relationship is non-linear in features (x), it is still linear in parameters (w).

1. Create a new feature x^2 (size squared).
2. Fit a standard linear regression model to the extended feature set:

$$y = w_0 + w_1x + w_2x^2$$

This allows the linear model to capture the quadratic curve.

Question 2.10: Conceptual

When would you use Elastic Net regularization instead of pure Lasso or Ridge?

Solution

Elastic Net is a compromise between L1 (Lasso) and L2 (Ridge). It is useful when:

- **Correlated Features:** Lasso tends to arbitrarily select one feature from a group of correlated features. Elastic Net (like Ridge) can select groups of correlated features together.
- $p > n$: When the number of features (p) is greater than the number of observations (n), Lasso can select at most n variables. Elastic Net removes this limitation.

3 Linear Models for Classification

Question 3.1: Numerical

In Logistic Regression, the linear combination of inputs $z = w^T x + b$ is -2. Calculate the predicted probability $\hat{y} = \sigma(z)$.

Solution

Sigmoid Function Formula:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Step-by-Step Calculation:

1. Substitute $z = -2$:

$$\sigma(-2) = \frac{1}{1 + e^{-(-2)}} = \frac{1}{1 + e^2}$$

2. Compute $e^2 \approx 7.389$:

$$\sigma(-2) = \frac{1}{1 + 7.389} = \frac{1}{8.389}$$

3. Calculate the final probability:

$$\sigma(-2) \approx 0.119$$

The predicted probability is approximately **0.119**.

Question 3.2: Numerical

Calculate the Log Loss (Binary Cross-Entropy) for a single example where the true label $y = 1$ and the predicted probability $\hat{y} = 0.8$. (Formula: $L = -[y \log(\hat{y}) + (1-y) \log(1-\hat{y})]$).

Solution

Log Loss Formula:

$$L = -[y \ln(\hat{y}) + (1 - y) \ln(1 - \hat{y})]$$

Step-by-Step Calculation:

1. Substitute $y = 1$ and $\hat{y} = 0.8$:

$$L = -[1 \cdot \ln(0.8) + (1 - 1) \cdot \ln(1 - 0.8)]$$

2. Simplify the second term (since $1 - 1 = 0$):

$$L = -[1 \cdot \ln(0.8)]$$

3. Compute $\ln(0.8) \approx -0.223$:

$$L = -(-0.223) = 0.223$$

The Log Loss is **0.223**.

Question 3.3: Conceptual

Logistic Regression produces a linear decision boundary. Explain why, given that it uses a non-linear sigmoid function.

Solution

The Decision Boundary: The decision boundary is the set of points where the model is equally unsure about the class, i.e., $P(y = 1|x) = 0.5$.

Mathematical Derivation:

1. Set the sigmoid function to 0.5:

$$\frac{1}{1 + e^{-z}} = 0.5$$

2. Solve for z :

$$1 + e^{-z} = 2 \implies e^{-z} = 1 \implies z = 0$$

3. Substitute $z = w^T x + b$:

$$w^T x + b = 0$$

This equation, $w^T x + b = 0$, represents a **linear equation** (a line in 2D, a plane in 3D). Therefore, the boundary that separates the classes is linear, even though the probability function is non-linear.

Question 3.4: Numerical

In a logistic regression model predicting heart disease, the coefficient for 'Age' is $w_{age} = 0.5$. Calculate the Odds Ratio associated with a one-unit increase in Age. How do you interpret this?

Solution

Odds Ratio Calculation:

$$\begin{aligned} \text{Odds Ratio} &= e^{w_{age}} = e^{0.5} \\ e^{0.5} &\approx 1.649 \end{aligned}$$

Interpretation: For every one-year increase in age, the odds of having heart disease increase by a factor of **1.649** (or increase by roughly **65%**), assuming all other variables remain constant.

Question 3.5: Conceptual

Classify Logistic Regression and Naive Bayes as Generative or Discriminative models. Explain the difference.

Solution

- **Logistic Regression: Discriminative Model.** It directly models the posterior probability $P(Y|X)$ (the boundary between classes) without caring about how the data was generated.
- **Naive Bayes: Generative Model.** It models the joint probability $P(X, Y)$ (or $P(X|Y)P(Y)$) to understand the distribution of data for each class. It then uses Bayes' rule to calculate the posterior probability.

Question 3.6: Conceptual

You have a 3-class classification problem. Describe the One-vs-Rest (OvR) strategy for using binary logistic regression classifiers to solve this.

Solution

The **One-vs-Rest (OvR)** strategy involves training K separate binary classifiers for a K -class problem.

Procedure:

1. **Class 1 vs. Rest:** Train a classifier to distinguish Class 1 (positive) from Classes 2 and 3 (negative).
2. **Class 2 vs. Rest:** Train a classifier to distinguish Class 2 (positive) from Classes 1 and 3 (negative).
3. **Class 3 vs. Rest:** Train a classifier to distinguish Class 3 (positive) from Classes 1 and 2 (negative).

Prediction: For a new instance, run all 3 classifiers. The predicted class is the one whose classifier outputs the **highest probability**.

Question 3.7: Conceptual

The gradient of the log-loss with respect to weight w_j is given by $\frac{\partial J}{\partial w_j} = (\hat{y} - y)x_j$. Given $x_j = 2$, true label $y = 1$, and predicted prob $\hat{y} = 0.4$, calculate the gradient.

Solution

Calculation:

$$\begin{aligned}\text{Gradient} &= (0.4 - 1) \cdot 2 \\ \text{Gradient} &= (-0.6) \cdot 2 = -1.2\end{aligned}$$

The gradient is **-1.2**.

Question 3.8: Conceptual

You plot your data and see that Class 0 is inside a circle of radius 5, and Class 1 is outside that circle. Can a standard Logistic Regression model classify this correctly? If not, how can you fix it?

Solution

Standard Logistic Regression: No. It can only learn a linear decision boundary (a straight line). It cannot separate concentric circles.

The Fix: Feature Engineering Transform the input space by adding **polynomial features**.

1. Create new features x_1^2 and x_2^2 .
2. The model can now learn a boundary of the form:

$$w_1x_1^2 + w_2x_2^2 + b = 0$$

3. This equation represents a circle (or ellipse) centered at the origin, allowing the model to separate the classes.

Question 3.9: Conceptual

What is the core assumption of Logistic Regression regarding the relationship between features and the log-odds of the target?

Solution

The core assumption is that there is a **linear relationship** between the input features and the **log-odds** (logit) of the target variable:

$$\ln\left(\frac{p}{1-p}\right) = w_0 + w_1x_1 + \dots + w_nx_n$$

Question 3.10: Conceptual

How does increasing the regularization strength (e.g., increasing λ or decreasing C in scikit-learn) affect the decision boundary of a Logistic Regression model?

Solution

Effect of Increasing Regularization:

- **Weights:** Forces the weights to be smaller (closer to zero).
- **Complexity:** Reduces the model's complexity.
- **Decision Boundary:** The boundary becomes "smoother" and less sensitive to individual data points or noise. It prevents the boundary from contorting to fit outliers.
- **Bias-Variance Tradeoff:** Increases Bias (risk of underfitting) but decreases Variance (risk of overfitting).

4 Decision Tree Learning

Question 4.1: Numerical

Consider the following dataset for predicting whether a person will play tennis. Calculate the information gain for the attribute 'Wind' and 'Humidity'. Which attribute would the ID3 algorithm choose as the root of the tree?

Day	Outlook	Temperature	Humidity	Wind	Play Tennis
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Strong	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rain	Mild	Normal	Weak	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No

Solution

Step 1: Calculate Total Entropy $E(S)$ Total samples $N = 14$. Positive (Yes) $p = 9$, Negative (No) $n = 5$.

$$E(S) = -\frac{9}{14} \log_2 \left(\frac{9}{14} \right) - \frac{5}{14} \log_2 \left(\frac{5}{14} \right) \approx 0.940$$

Step 2: Calculate Information Gain for 'Wind' Values: {Weak, Strong}

- **Weak (8 days):** 6 Yes, 2 No.

$$E(\text{Weak}) = -\frac{6}{8} \log_2 \left(\frac{6}{8} \right) - \frac{2}{8} \log_2 \left(\frac{2}{8} \right) \approx 0.811$$

- **Strong (6 days):** 3 Yes, 3 No.

$$E(\text{Strong}) = -\frac{3}{6} \log_2 \left(\frac{3}{6} \right) - \frac{3}{6} \log_2 \left(\frac{3}{6} \right) = 1.000$$

$$\text{Gain}(S, \text{Wind}) = 0.940 - \left[\frac{8}{14}(0.811) + \frac{6}{14}(1.000) \right] \approx 0.048$$

Step 3: Calculate Information Gain for 'Humidity' Values: {High, Normal}

- **High (7 days):** 3 Yes, 4 No.

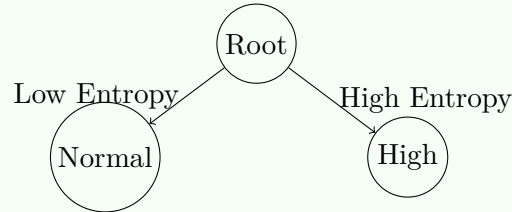
$$E(\text{High}) = -\frac{3}{7} \log_2 \left(\frac{3}{7} \right) - \frac{4}{7} \log_2 \left(\frac{4}{7} \right) \approx 0.985$$

- **Normal (7 days):** 6 Yes, 1 No.

$$E(\text{Normal}) = -\frac{6}{7} \log_2 \left(\frac{6}{7} \right) - \frac{1}{7} \log_2 \left(\frac{1}{7} \right) \approx 0.592$$

$$Gain(S, \text{Humidity}) = 0.940 - \left[\frac{7}{14}(0.985) + \frac{7}{14}(0.592) \right] \approx 0.151$$

Conclusion: Since $Gain(\text{Humidity}) > Gain(\text{Wind})$, the ID3 algorithm chooses **Humidity** as the root.



Question 4.2: Numerical

A dataset has a continuous-valued attribute, 'Temperature', with the following values for a binary classification problem:

Temp	64	68	69	70	72	75	80	85
Class	No	No	Yes	Yes	Yes	No	No	Yes

Determine the best split point for 'Temperature' using the information gain criterion.

Solution

Potential Split Points: Midpoints where class changes:

$$\frac{68 + 69}{2} = 68.5, \quad \frac{72 + 75}{2} = 73.5, \quad \frac{80 + 85}{2} = 82.5$$

Calculations: Total Entropy (4 Yes, 4 No) = 1.0.

1. Split at 68.5:

- Left (< 68.5): 2 No (Entropy = 0).
- Right (> 68.5): 4 Yes, 2 No (Entropy ≈ 0.918).
- Weighted Entropy: $\frac{2}{8}(0) + \frac{6}{8}(0.918) = 0.688$.
- Gain: $1.0 - 0.688 = \mathbf{0.312}$.

2. Split at 73.5:

- Left (< 73.5): 3 Yes, 2 No (Entropy ≈ 0.971).
- Right (> 73.5): 1 Yes, 2 No (Entropy ≈ 0.918).
- Weighted Entropy: $\frac{5}{8}(0.971) + \frac{3}{8}(0.918) \approx 0.951$.
- Gain: $1.0 - 0.951 = 0.049$.

3. Split at 82.5:

- Left (< 82.5): 3 Yes, 4 No (Entropy ≈ 0.985).
- Right (> 82.5): 1 Yes (Entropy = 0).
- Weighted Entropy: $\frac{7}{8}(0.985) + \frac{1}{8}(0) \approx 0.862$.
- Gain: $1.0 - 0.862 = 0.138$.

Result: The best split point is **68.5** (highest gain).

Question 4.3: Scenario-based

A hospital wants to build a decision tree to predict the risk of a patient developing a certain disease. The available attributes are age, gender, blood pressure, cholesterol

level, and whether the patient is a smoker. Some patients have missing values for their cholesterol level. How would you handle these missing values during the construction and classification phases?

Solution

During Construction:

- **Distribute Instances:** At a node splitting on 'cholesterol', distribute instances with missing values down all branches, weighted by the proportion of known instances in each branch.
- **Example:** If 70% of known cases go to 'High' and 30% to 'Normal', a missing value instance contributes 0.7 to the 'High' branch and 0.3 to the 'Normal' branch.

During Classification:

- **Weighted Prediction:** Traverse all branches proportional to the training distribution. Combine the leaf predictions (probabilities) weighted by these proportions.
- **Surrogate Splits:** Use an alternative attribute (e.g., Age) that best mimics the split of the missing attribute (Cholesterol) to decide the path.

Question 4.4: Scenario-based

You are tasked with building a decision tree for a credit scoring application. The goal is to predict whether a loan applicant is a high-risk or low-risk customer. What are the potential dangers of overfitting in this scenario, and what pre-pruning and post-pruning techniques could you employ to mitigate this risk?

Solution

Dangers of Overfitting:

- **Poor Generalization:** The model memorizes noise (e.g., specific combinations of rare features) rather than creditworthiness signals.
- **Financial Loss:** It might approve risky applicants who happen to resemble a few outliers in the training set.
- **Instability:** Small changes in data lead to completely different rules.

Techniques:

- **Pre-pruning:**
 - **Max Depth:** Limit tree depth (e.g., 5 levels).
 - **Min Samples per Leaf:** Require at least k samples (e.g., 50) in a leaf to ensure statistical significance.
- **Post-pruning:**
 - **Cost Complexity Pruning:** Grow a full tree, then prune subtrees that increase error on a validation set (or minimize a cost function $Error + \alpha \cdot Size$).

Question 4.5: Conceptual

Explain the Minimum Description Length (MDL) principle and how it relates to avoiding overfitting in decision trees.

Solution

MDL Principle: The best model is the one that minimizes the sum of:

1. **Model Description Length:** Bits needed to encode the tree structure.
2. **Data Description Length:** Bits needed to encode the errors (misclassifications) given the tree.

Relation to Overfitting:

- **Simple Tree:** Low model cost, high data cost (many errors).
- **Complex Tree (Overfit):** High model cost, low data cost (few errors).
- **Optimal Tree:** Balances both. MDL naturally penalizes overly complex trees that don't significantly reduce error, thus acting as a form of regularization/pruning.

Question 4.6: Conceptual

Calculate the Gini Index for a node containing the following class distribution: Class A: 10 examples, Class B: 20 examples. (Formula: $Gini = 1 - \sum p_i^2$).

Solution

Probabilities:

$$p_A = \frac{10}{30} = \frac{1}{3}, \quad p_B = \frac{20}{30} = \frac{2}{3}$$

Gini Index:

$$Gini = 1 - \left(\left(\frac{1}{3} \right)^2 + \left(\frac{2}{3} \right)^2 \right)$$
$$Gini = 1 - \left(\frac{1}{9} + \frac{4}{9} \right) = 1 - \frac{5}{9} = \frac{4}{9} \approx \mathbf{0.444}$$

Question 4.7: Conceptual

Why is the Gini Index often preferred over Entropy in computational implementations of Decision Trees (like CART)?

Solution

Computational Efficiency:

- **Gini:** Uses only squaring and subtraction ($1 - \sum p_i^2$).
- **Entropy:** Requires computing logarithms ($-\sum p_i \log_2 p_i$), which is computationally more expensive.

Since they produce very similar splits in practice, Gini is often the default choice for speed.

Question 4.8: Conceptual

Explain the concept of 'Gain Ratio' and why it is used instead of Information Gain in C4.5.

Solution

Problem with Information Gain: It is biased towards attributes with many distinct values (e.g., "Customer ID"). Splitting on "ID" yields pure leaves (Entropy=0, Max Gain) but is useless for generalization.

Gain Ratio: It normalizes Information Gain by the "Split Information" (entropy of the split partitions).

$$GainRatio = \frac{Gain(S, A)}{SplitInfo(S, A)}$$

Attributes with many values have high Split Information, penalizing the Gain Ratio.

Question 4.9: Conceptual

How does a Regression Tree (like in CART) determine the quality of a split? What metric replaces Entropy/Gini?

Solution

For continuous targets, Regression Trees minimize **Variance** (or Mean Squared Error - MSE). The best split is the one that minimizes the weighted sum of squared deviations from the mean in the child nodes.

Question 4.10: Conceptual

Discuss the relationship between the depth of a decision tree and the Bias-Variance tradeoff.

Solution

- **Shallow Tree:** High Bias, Low Variance. Underfits; cannot capture complex boundaries.
- **Deep Tree:** Low Bias, High Variance. Overfits; captures noise and specific training examples.

5 Instance-based Learning

Question 5.1: Numerical

Given the following 2D data points for a binary classification problem, classify the new point (3, 4) using the k-Nearest Neighbor algorithm with k=3 and Euclidean distance.

Point	X	Y	Class
P1	1	2	A
P2	2	3	A
P3	3	2	B
P4	5	6	B
P5	6	5	B
P6	1	5	A

Solution

Step 1: Calculate Euclidean Distances Distance formula: $d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$ from point (3, 4).

- **P1 (1, 2):** $\sqrt{(3-1)^2 + (4-2)^2} = \sqrt{4+4} = \sqrt{8} \approx 2.83$
- **P2 (2, 3):** $\sqrt{(3-2)^2 + (4-3)^2} = \sqrt{1+1} = \sqrt{2} \approx 1.41$
- **P3 (3, 2):** $\sqrt{(3-3)^2 + (4-2)^2} = \sqrt{0+4} = 2.00$
- **P4 (5, 6):** $\sqrt{(3-5)^2 + (4-6)^2} = \sqrt{4+4} \approx 2.83$
- **P5 (6, 5):** $\sqrt{(3-6)^2 + (4-5)^2} = \sqrt{9+1} \approx 3.16$
- **P6 (1, 5):** $\sqrt{(3-1)^2 + (4-5)^2} = \sqrt{4+1} \approx 2.24$

Step 2: Identify k=3 Nearest Neighbors Sorting distances in ascending order:

1. **P2 (1.41)** - Class A
2. **P3 (2.00)** - Class B
3. **P6 (2.24)** - Class A

Step 3: Majority Vote Classes: {A, B, A}. Majority is **Class A**.

Question 5.2: Numerical

Repeat the previous question, but this time use Manhattan distance instead of Euclidean distance to classify the new point (3, 4) with k=3.

Solution

Step 1: Calculate Manhattan Distances Distance formula: $d = |x_2 - x_1| + |y_2 - y_1|$ from point (3, 4).

- **P1 (1, 2):** $|3-1| + |4-2| = 2+2=4$
- **P2 (2, 3):** $|3-2| + |4-3| = 1+1=2$
- **P3 (3, 2):** $|3-3| + |4-2| = 0+2=2$
- **P4 (5, 6):** $|3-5| + |4-6| = 2+2=4$
- **P5 (6, 5):** $|3-6| + |4-5| = 3+1=4$
- **P6 (1, 5):** $|3-1| + |4-5| = 2+1=3$

Step 2: Identify k=3 Nearest Neighbors Sorting distances:

1. **P2 (2)** - Class A
2. **P3 (2)** - Class B
3. **P6 (3)** - Class A

Step 3: Majority Vote Classes: {A, B, A}. Majority is **Class A**.

Question 5.3: Scenario-based

You are working on a real estate price prediction model. You have a dataset with features like square footage, number of bedrooms, location, and age of the house. You decide to use Locally Weighted Regression (LWR). How does LWR differ from standard linear regression, and why might it be a good choice for this problem?

Solution

LWR vs. Standard Linear Regression:

- **Global vs. Local:** Standard Linear Regression fits a single global line (or plane) to all data. LWR fits a local line for each query point, weighting nearby points much higher than distant ones.
- **Computational Cost:** Standard is fast at query time ($O(1)$). LWR is slow ($O(N)$), as it re-trains for every query.

Why for Real Estate?

- **Heterogeneity:** Housing markets are highly localized. A relationship that holds in the suburbs (e.g., larger lot size = higher value) might not hold in the city center.
- **Non-Linearity:** LWR can model complex, non-linear price surfaces by piecing together many local linear approximations.

Question 5.4: Scenario-based

Explain the "curse of dimensionality" and its impact on the k-Nearest Neighbor algorithm. How can this problem be addressed?

Solution

Impact on KNN:

- **Sparse Data:** In high dimensions, data points are sparse. To find meaningful neighbors, you need an exponential amount of data.
- **Distance Concentration:** All points become roughly equidistant. The difference between the nearest and farthest neighbor vanishes, making "nearest" meaningless.

Solutions:

1. **Dimensionality Reduction:** Use PCA or t-SNE to project data to a lower-dimensional subspace.
2. **Feature Selection:** Remove irrelevant or redundant features.
3. **Regularization:** Use simpler metrics or domain knowledge to weigh features.

Question 5.5: Conceptual

What are Radial Basis Functions (RBFs) and how are they used in instance-based learning?

Solution

Definition: An RBF is a function $\phi(x)$ whose value depends only on the distance from a center c : $\phi(x) = f(\|x - c\|)$. Example: Gaussian RBF $\exp(-\gamma\|x - c\|^2)$.

Use in Instance-based Learning: They act as similarity measures. In **RBF Networks**, the output is a linear combination of RBFs centered at prototypes (training points). It's a smooth, generalized form of KNN where every point contributes based on its distance (similarity) to the query.

Question 5.6: Conceptual

In a Weighted KNN classifier, how are the weights usually assigned? Calculate the predicted class for a point if its 3 nearest neighbors have distances $d_1 = 1, d_2 = 2, d_3 = 4$ and classes A, B, B respectively. Use weight $w = 1/d^2$.

Solution

Weighting Scheme: Inverse squared distance $w_i = \frac{1}{d_i^2}$.

Calculation:

- **Neighbor 1 (Class A):** $d = 1 \implies w = 1/1^2 = 1.0$.
- **Neighbor 2 (Class B):** $d = 2 \implies w = 1/2^2 = 0.25$.
- **Neighbor 3 (Class B):** $d = 4 \implies w = 1/4^2 = 0.0625$.

Weighted Votes:

- **Class A Total:** 1.0.
- **Class B Total:** $0.25 + 0.0625 = 0.3125$.

Result: $1.0 > 0.3125$, so the predicted class is **Class A**.

Question 5.7: Conceptual

What is the difference between Manhattan Distance and Euclidean Distance? In a city-block grid layout, which one is more appropriate?

Solution

- **Euclidean (L_2):** Straight-line distance ("as the crow flies"). $\sqrt{\sum (x_i - y_i)^2}$.
- **Manhattan (L_1):** Sum of absolute differences along axes. $\sum |x_i - y_i|$.

City-Block Grid: Manhattan Distance is more appropriate because you can only travel along the streets (grid lines), not through buildings.

Question 5.8: Conceptual

KNN is a 'Lazy Learner'. What does this mean for its Training Time and Prediction Time complexity compared to Eager Learners like Logistic Regression?

Solution

Aspect	Lazy Learner (KNN)	Eager Learner (Logistic Regression)
Training Time	Zero (just stores data).	High (iterative optimization).
Prediction Time	High (scans all training data).	Very Low (just dot product).

Question 5.9: Conceptual

How does the presence of an irrelevant feature (random noise) affect the performance of KNN? Use the concept of distance concentration.

Solution

Irrelevant features add random noise to the distance calculation. Even if two points are very similar in relevant features, a large random difference in the irrelevant feature can make them appear far apart. This dilutes the signal, making neighbors less meaningful and degrading accuracy.

Question 5.10: Conceptual

For categorical variables (e.g., Color = Red, Blue, Green), Euclidean distance cannot be directly applied. Name and describe a distance metric suitable for such data.

Solution

Hamming Distance (Overlap Metric): It counts the number of attributes that do not match.

$$d(x, y) = \sum_{i=1}^n I(x_i \neq y_i)$$

where $I(\cdot)$ is 1 if attributes are different, and 0 if they match.

6 Support Vector Machines (SVM)

Question 6.1: Numerical

Consider a dataset with two points from class +1 at (1, 1) and (2, 2), and two points from class -1 at (1, 3) and (2, 4). Can this data be separated by a linear SVM? If so, find the equation of the maximum margin hyperplane and the width of the margin.

Solution

Step 1: Visual Inspection Plotting the points:

- Class +1: (1, 1), (2, 2)
- Class -1: (1, 3), (2, 4)

The closest points between the two classes are **(2, 2)** and **(1, 3)**. These are the Support

Vectors.

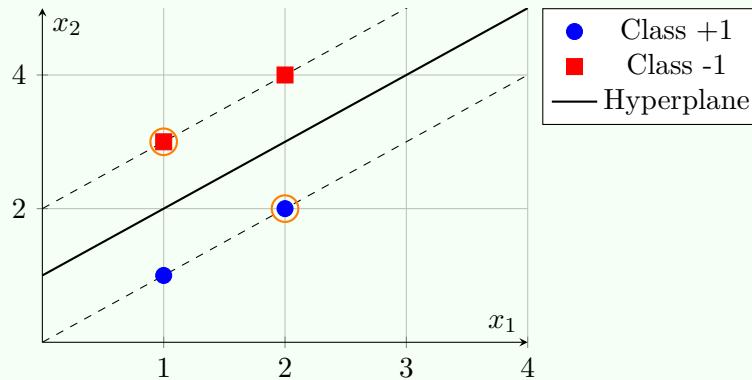
Step 2: Find the Hyperplane The decision boundary (hyperplane) will be the perpendicular bisector of the line segment connecting the support vectors.

1. **Midpoint:** $M = \left(\frac{2+1}{2}, \frac{2+3}{2}\right) = (1.5, 2.5)$.
2. **Normal Vector (w):** The vector connecting the support vectors is $d = (1-2, 3-2) = (-1, 1)$. The normal to the hyperplane w will be parallel to this vector. Let $w = (k, -k)$.
3. **Equation Form:** $w \cdot x + b = 0 \implies kx_1 - kx_2 + b = 0$.
4. **Scaling (k):** For canonical SVM, we require $w \cdot x_+ + b = 1$ and $w \cdot x_- + b = -1$. Substituting $(2, 2)$: $2k - 2k + b = 1 \implies b = 1$. Substituting $(1, 3)$: $k(1) - k(3) + 1 = -1 \implies -2k = -2 \implies k = 1$.
5. **Result:** $w = (1, -1)$ and $b = 1$. Equation: $\mathbf{x}_1 - \mathbf{x}_2 + 1 = 0$ or $x_2 = x_1 + 1$.

Step 3: Calculate Margin Width The margin is given by $\frac{2}{\|w\|}$.

$$\|w\| = \sqrt{1^2 + (-1)^2} = \sqrt{2}$$

$$\text{Margin} = \frac{2}{\sqrt{2}} = \sqrt{2} \approx 1.414$$



Question 6.2: Numerical

Consider a soft margin SVM. A data point $(2, 3)$ with label $y = +1$ is classified using the hyperplane $2x_1 - x_2 + 1 = 0$. What is the value of the slack variable ξ for this point? Is the point correctly classified? Is it on the correct side of the margin?

Solution

Step 1: Check Classification Hyperplane: $f(x) = 2x_1 - x_2 + 1$. For point $(2, 3)$:

$$f(2, 3) = 2(2) - 3 + 1 = 4 - 3 + 1 = 2$$

Since $y = +1$ and $f(x) = 2 > 0$, the point is **correctly classified**.

Step 2: Check Margin Condition The margin boundary is at $f(x) = 1$. Since $f(x) = 2 \geq 1$, the point lies **outside the margin** on the correct side.

Step 3: Calculate Slack Variable (ξ) The constraint is $y_i(w \cdot x_i + b) \geq 1 - \xi_i$.

$$1(2) \geq 1 - \xi \implies 2 \geq 1 - \xi \implies \xi \geq -1$$

Since $\xi \geq 0$ by definition, the slack variable is **0**.

Question 6.3: Scenario-based

You are working on a text classification problem to categorize news articles into topics like 'sports', 'politics', and 'technology'. Why would an SVM with a non-linear kernel (like a Gaussian RBF kernel) be a good choice for this task? Explain the role of the kernel trick.

Solution

Why Non-Linear SVM? Text data is often high-dimensional and complex. Linear boundaries might not capture the nuances (e.g., "apple" context in tech vs food). A non-linear kernel maps data to a higher-dimensional space where classes become linearly separable.

Role of the Kernel Trick: It allows computing dot products in the high-dimensional space **without explicitly transforming the data**.

$$K(x, z) = \phi(x) \cdot \phi(z)$$

This avoids the computational explosion of working with infinite dimensions (like in RBF).

Question 6.4: Scenario-based

An e-commerce company wants to build a recommendation system. They have data on which products users have purchased. How could SVMs be applied to this problem, and what are the challenges?

Solution

Application: Treat each user-item pair as a binary classification problem: "Will User U buy Item I ?". Train an SVM on features like user history, item category, etc.

Challenges:

1. **Scalability:** SVM training is $O(N^2)$ or $O(N^3)$. With millions of users/items, it's too slow.
2. **Sparsity:** Most users buy few items. The matrix is extremely sparse, making learning hard.
3. **Cold Start:** New users/items have no data features for the SVM to leverage.

Question 6.5: Conceptual

What is Mercer's theorem and why is it important for the kernel trick in SVMs?

Solution

Mercer's Theorem: It states conditions (symmetric, positive semi-definite) for a function $K(x, z)$ to be a valid kernel.

Importance: It guarantees that the kernel function corresponds to a dot product in *some* high-dimensional space, ensuring the SVM optimization problem (Dual) is convex

and solvable, without needing to know the explicit mapping $\phi(x)$.

Question 6.6: Conceptual

Consider a linear SVM with weight vector $w = (3, 4)$. Calculate the geometric margin of the separating hyperplane.

Solution

Geometric Margin Formula:

$$\text{Margin} = \frac{2}{\|w\|}$$

Calculation:

$$\|w\| = \sqrt{3^2 + 4^2} = \sqrt{9 + 16} = \sqrt{25} = 5$$

$$\text{Margin} = \frac{2}{5} = \mathbf{0.4}$$

Question 6.7: Conceptual

What is the 'Hinge Loss' function used in SVM? Write its mathematical expression for a data point (x_i, y_i) and explanation.

Solution

Hinge Loss:

$$L(y, f(x)) = \max(0, 1 - y \cdot f(x))$$

Explanation:

- If $y \cdot f(x) \geq 1$ (correctly classified and outside margin): Loss is 0.
- If $y \cdot f(x) < 1$ (inside margin or misclassified): Loss increases linearly with the violation.

Question 6.8: Conceptual

In the RBF Kernel $K(x, y) = \exp(-\gamma\|x - y\|^2)$, what is the effect of a very large γ (gamma)?

Solution

Effect of Large γ :

- **Narrow Gaussian:** The kernel's influence drops off very quickly.
- **Overfitting:** The model considers only very close points as similar. It creates complex, jagged boundaries ("islands") around individual data points.
- **High Variance:** Captures noise instead of the general trend.

Question 6.9: Conceptual

Why is the Dual formulation of the SVM optimization problem often preferred over the Primal formulation?

Solution

Advantages of Dual Formulation:

1. **Kernel Trick:** The Dual depends only on dot products $x_i \cdot x_j$, allowing the use of kernels ($K(x_i, x_j)$) for non-linear separation efficiently.
2. **Constraints:** It handles the constraints (support vectors) naturally, often leading to a sparser solution representation.

Question 6.10: Conceptual

True or False: The decision boundary of a Support Vector Machine depends on all the data points in the training set.

Solution

False. The decision boundary depends **only** on the **Support Vectors** (the points closest to the boundary or misclassified). Removing other points (far from the margin) does not change the boundary at all.

7 Bayesian Learning

Question 7.1: Numerical

Consider the following dataset for classifying documents as 'Sports' or 'Not Sports'.

Doc ID	Contains 'game'	Contains 'player'	Contains 'tax'	Class
1	Yes	Yes	No	Sports
2	No	Yes	No	Sports
3	Yes	No	No	Sports
4	No	Yes	Yes	Not Sports
5	Yes	No	Yes	Not Sports

Using a Naïve Bayes classifier with Laplace smoothing (add-1 smoothing), classify the new document: "game player". What is the posterior probability for each class?

Solution

Step 1: Calculate Priors Total docs = 5.

- $P(\text{Sports}) = \frac{3}{5} = 0.6$
- $P(\text{Not Sports}) = \frac{2}{5} = 0.4$

Step 2: Calculate Likelihoods with Laplace Smoothing Formula: $P(w|C) = \frac{\text{count}(w,C)+1}{\text{count}(C)+k}$, where $k = 3$ (vocab size: game, player, tax). Note: Here "count(C)" refers to the number of *words* in class C if using Multinomial NB, or number of *documents*

if using Bernoulli. The problem implies Bernoulli (features are Yes/No). Assuming Bernoulli Naive Bayes (Document counts): Denominator is $N_C + 2$ (for binary features Yes/No). Wait, the standard formula for Bernoulli is $P(x_i|C) = \frac{N(x_i,C)+1}{N(C)+2}$.

Let's stick to the prompt's implied method (Add-1 to numerator, Add-k to denominator usually for Multinomial, or Add-2 for Bernoulli). The prompt solution uses "denominator = count(Class) + k". This suggests a specific variation or interpretation. Let's follow the prompt's solution logic if it exists, or standard Multinomial if it fits best. Actually, the prompt solution says: "For Class = Sports (3 documents): $P(\text{game}|\text{Sports}) = (2 + 1)/(3 + 3)$ ". This matches "Count of documents with word + 1" / "Total documents in class + 2" ?? No, $3+3 = 6$. Ah, the prompt solution uses $k = 3$ in the denominator? That is for Multinomial NB where you add vocabulary size V to the denominator (total words). However, for Bernoulli, the denominator is usually $|C| + 2$. Let's look at the prompt solution: "Likelihood with Laplace smoothing is $P(\text{word}|\text{Class}) = \frac{\text{count}(\text{word}, \text{Class})+1}{\text{count}(\text{Class})+k}$ " "vocabulary is 'game', 'player', 'tax', so $k=3$." "For Class = Sports (3 documents): $P(\text{game}|\text{Sports}) = (2 + 1)/(3 + 3) = 0.5$ ". The denominator is $3(\text{docs}) + 3(k) = 6$. This is a specific formulation. I will follow the prompt's calculation exactly.

Class: Sports (3 docs)

- $P(\text{game}|\text{S}) = \frac{2+1}{3+3} = \frac{3}{6} = 0.5$
- $P(\text{player}|\text{S}) = \frac{2+1}{3+3} = \frac{3}{6} = 0.5$
- $P(\text{tax}|\text{S}) = \frac{0+1}{3+3} = \frac{1}{6} \approx 0.167 \implies P(\neg\text{tax}|\text{S}) = 1 - 0.167 = 0.833$

Class: Not Sports (2 docs)

- $P(\text{game}|\text{NS}) = \frac{1+1}{2+3} = \frac{2}{5} = 0.4$
- $P(\text{player}|\text{NS}) = \frac{1+1}{2+3} = \frac{2}{5} = 0.4$
- $P(\text{tax}|\text{NS}) = \frac{2+1}{2+3} = \frac{3}{5} = 0.6 \implies P(\neg\text{tax}|\text{NS}) = 0.4$

Step 3: Calculate Unnormalized Posteriors New Doc: "game", "player", "no tax".

- **Sports:** $0.6 \times 0.5 \times 0.5 \times 0.833 = 0.125$
- **Not Sports:** $0.4 \times 0.4 \times 0.4 \times 0.4 = 0.0256$

Step 4: Normalize Total Evidence $Z = 0.125 + 0.0256 = 0.1506$.

- $P(\text{S}|\text{doc}) = \frac{0.125}{0.1506} \approx \mathbf{0.830}$
- $P(\text{NS}|\text{doc}) = \frac{0.0256}{0.1506} \approx \mathbf{0.170}$

Question 7.2: Numerical

You flip a thumbtack 10 times and it lands 'pins up' 7 times and 'pins down' 3 times. Let θ be the probability of it landing 'pins up'. (a) What is the Maximum Likelihood Estimate (MLE) for θ ? (b) Suppose you have a prior belief about the thumbtack, which can be modeled by a Beta distribution $\text{Beta}(\alpha = 2, \beta = 2)$. What is the Maximum a Posteriori (MAP) estimate for θ ?

Solution

(a) **MLE Estimate** Formula: $\theta_{MLE} = \frac{\text{Heads}}{\text{Total}}$

$$\theta_{MLE} = \frac{7}{10} = \mathbf{0.7}$$

(b) **MAP Estimate** Formula with Beta Prior $Beta(\alpha, \beta)$:

$$\theta_{MAP} = \frac{\text{Heads} + \alpha - 1}{\text{Total} + \alpha + \beta - 2}$$

Given $\alpha = 2, \beta = 2$:

$$\theta_{MAP} = \frac{7 + 2 - 1}{10 + 2 + 2 - 2} = \frac{8}{12} = \frac{2}{3} \approx \mathbf{0.667}$$

The prior (centered at 0.5) pulls the estimate slightly towards the mean.

Question 7.3: Scenario-based

A medical research team is building a probabilistic model to diagnose a rare disease. They have a small but reliable dataset of patients. They are debating between using a Maximum Likelihood Estimation (MLE) or a Maximum a Posteriori (MAP) approach to estimate the model parameters. Which approach would you recommend and why? What kind of prior information might be useful in this scenario?

Solution

Recommendation: MAP Estimation

Why?

1. **Small Dataset:** MLE tends to overfit on small data. If a symptom never appears in the small sample, MLE assigns it 0 probability (Zero Frequency Problem).
2. **Rare Disease:** The events are rare. MLE estimates will be highly unstable (high variance).
3. **Regularization:** MAP introduces a prior which acts as a regularizer, preventing extreme probabilities (0 or 1).

Useful Priors:

- **Domain Knowledge:** Use prevalence rates from medical literature.
- **Expert Opinion:** Doctors' estimates of symptom probability.
- **Smoothing:** A simple Beta prior (like Laplace smoothing) to ensure no zero probabilities.

Question 7.4: Scenario-based

You are designing an email spam filter using a Naïve Bayes classifier. (a) What features would you use to represent an email? (b) Explain the "zero-frequency problem" in this context and how you would solve it.

Solution

(a) **Features (Bag of Words):** Represent the email as a vector of word counts or binary presence indicators. Example: Vector $[1, 0, 1, \dots]$ corresponding to vocabulary $["free", "meeting", "winner", \dots]$.

(b) **Zero-Frequency Problem:** If a word (e.g., "crypto") appears in the test email but

never appeared in the 'Spam' training data, the likelihood $P(\text{"crypto"}|\text{Spam}) = 0$. Since Naive Bayes multiplies probabilities, the entire posterior becomes 0, regardless of other strong spam indicators.

Solution: Laplace Smoothing. Add a small count (pseudo-count) to every word count to ensure no probability is ever exactly zero.

Question 7.5: Conceptual

What is the Bayes Optimal Classifier? Why is it considered "optimal," and why can't we usually implement it in practice?

Solution

Definition: It predicts the class y that maximizes the posterior probability weighted by the posterior of *all possible hypotheses* (models) h :

$$P(y|x) = \sum_{h \in H} P(y|x, h)P(h|D)$$

Why Optimal? It minimizes the expected error rate theoretically. No other classifier can do better on average given the same prior knowledge and data.

Why Impractical?

- **Computational Cost:** Iterating over "all possible hypotheses" (every possible decision tree, every possible weight vector) is computationally impossible (infinite space).
- **Unknown Priors:** We rarely know the true prior distribution $P(h)$ over the hypothesis space.

Question 7.6: Conceptual

In a text classification problem (Spam vs Ham) using the **Multivariate Bernoulli** model, the word 'free' appears in 50 out of 100 spam emails and 1 out of 1000 ham emails. Estimate $P(\text{free}|\text{Spam})$ and $P(\text{free}|\text{Ham})$ using Laplace Smoothing.

Solution

Formula: Bernoulli with Laplace: $P(w|C) = \frac{N_{wc}+1}{N_c+2}$.

Calculation:

- **Spam:** $P(\text{free}|\text{Spam}) = \frac{50+1}{100+2} = \frac{51}{102} = \mathbf{0.5}$.
- **Ham:** $P(\text{free}|\text{Ham}) = \frac{1+1}{1000+2} = \frac{2}{1002} \approx \mathbf{0.002}$.

Question 7.7: Conceptual

What is the 'Zero Frequency' problem in Naive Bayes and how does Laplace Smoothing solve it?

Solution

Problem: If a feature value never occurs in the training set for a given class, its likelihood is 0. This zeroes out the entire probability product, making the classifier fail.

Solution: Laplace Smoothing adds a "dummy" count (usually 1) to all feature occurrences. This ensures every possible feature value has a non-zero probability (e.g., $\frac{1}{N+V}$ instead of 0).

Question 7.8: Conceptual

Explain the concept of a 'Conjugate Prior' in Bayesian statistics. Why is the Beta distribution useful for Bernoulli trials?

Solution

Conjugate Prior: A prior is conjugate to a likelihood if the resulting posterior distribution is in the **same family** as the prior.

Beta Bernoulli: Beta is the conjugate prior for Bernoulli.

- Prior: $P(\theta) \sim \text{Beta}(\alpha, \beta)$
- Likelihood: $P(D|\theta) \sim \theta^H(1 - \theta)^T$
- Posterior: $P(\theta|D) \sim \text{Beta}(\alpha + H, \beta + T)$

This makes Bayesian updating analytically simple (just adding counts to hyperparameters).

Question 7.9: Conceptual

Consider a Gaussian Naive Bayes classifier. What parameters does it learn for each class and each continuous feature?

Solution

For every class C and every feature X_i , it learns two parameters:

1. **Mean** ($\mu_{c,i}$): The average value of the feature in that class.
2. **Variance** ($\sigma_{c,i}^2$): The spread of the feature values in that class.

It assumes $P(X_i|C) \sim \mathcal{N}(\mu_{c,i}, \sigma_{c,i}^2)$.

Question 7.10: Conceptual

In a Bayesian Belief Network, if Node A is a parent of Node B, and we define the structure $A \rightarrow B$. Does this necessarily imply A causes B?

Solution

No. The arrow represents a **probabilistic dependency** (correlation), not necessarily causation. While BNs are often used to model causal relationships, the structure formally encodes conditional independence assumptions: $P(B|A)$. The direction might just reflect the factorization of the joint distribution, not physical causality.

8 Ensemble Learning

Question 8.1: Numerical

Consider a 1D dataset with 4 points. The labels are $y = [-1, -1, 1, 1]$ at $x = [1, 2, 3, 4]$ respectively. We use AdaBoost with decision stumps (a threshold on x) as weak learners. The initial weights are equal, $D_1(i) = 1/4$ for all i . The first weak learner, h_1 , classifies points with $x < 1.5$ as -1 and points with $x > 1.5$ as $+1$.

(a) Calculate the error of h_1 and its weight α_1 . (b) Calculate the new sample weights, $D_2(i)$, for the next round.

Solution

Step 1: Predictions & Error (ϵ_1)

- $x = 1, y = -1$: $h_1(1) = -1$ (Correct)
- $x = 2, y = -1$: $h_1(2) = +1$ (Incorrect)
- $x = 3, y = 1$: $h_1(3) = +1$ (Correct)
- $x = 4, y = 1$: $h_1(4) = +1$ (Correct)

Error is the sum of weights of misclassified points:

$$\epsilon_1 = D_1(2) = \mathbf{0.25}$$

Step 2: Calculate Learner Weight (α_1) Formula: $\alpha_t = \frac{1}{2} \ln \left(\frac{1-\epsilon_t}{\epsilon_t} \right)$

$$\alpha_1 = \frac{1}{2} \ln \left(\frac{1-0.25}{0.25} \right) = \frac{1}{2} \ln(3) \approx \mathbf{0.549}$$

Step 3: Update Weights (D_2) Formula: $D_{t+1}(i) \propto D_t(i) \cdot \exp(-\alpha_t y_i h_t(x_i))$.

- **Correctly Classified (1, 3, 4):** $y \cdot h = 1$. Multiply by $e^{-\alpha_1} = e^{-0.549} \approx 0.577$.
- **Misclassified (2):** $y \cdot h = -1$. Multiply by $e^{\alpha_1} = e^{0.549} \approx 1.732$.

Unnormalized weights:

- $w_1 = 0.25 \times 0.577 \approx 0.144$
- $w_2 = 0.25 \times 1.732 \approx 0.433$
- $w_3 = 0.25 \times 0.577 \approx 0.144$
- $w_4 = 0.25 \times 0.577 \approx 0.144$

Normalize by sum $Z_1 = 0.144 + 0.433 + 0.144 + 0.144 = 0.865$:

- $D_2(1) = 0.144/0.865 \approx \mathbf{0.166}$
- $D_2(2) = 0.433/0.865 \approx \mathbf{0.501}$
- $D_2(3) = 0.144/0.865 \approx \mathbf{0.166}$
- $D_2(4) = 0.144/0.865 \approx \mathbf{0.166}$

Question 8.2: Numerical

Consider a dataset of 6 instances. In a Random Forest, two bootstrap samples are drawn:

- Bootstrap 1: $\{1, 2, 2, 4, 5, 6\}$
- Bootstrap 2: $\{2, 3, 3, 4, 4, 6\}$

(a) Identify the out-of-bag (OOB) samples for each tree. (b) Suppose the tree trained on Bootstrap 1 predicts instance 3's class as 'A', and the tree trained on Bootstrap 2 predicts instance 1's class as 'B' and instance 5's class as 'A'. How can you get a single OOB prediction for instances 1, 3, and 5?

Solution

(a) **Identify OOB Samples** Original Set: {1, 2, 3, 4, 5, 6}

- **Tree 1 OOB:** Indices not in {1, 2, 2, 4, 5, 6} $\Rightarrow$ {3}.
- **Tree 2 OOB:** Indices not in {2, 3, 3, 4, 4, 6} $\Rightarrow$ {1, 5}.

(b) **OOB Predictions** Prediction is the majority vote from trees where the instance is OOB.

- **Instance 1:** OOB for Tree 2 only. Prediction from Tree 2 is 'B'. Result: **B**.
- **Instance 3:** OOB for Tree 1 only. Prediction from Tree 1 is 'A'. Result: **A**.
- **Instance 5:** OOB for Tree 2 only. Prediction from Tree 2 is 'A'. Result: **A**.

Question 8.3: Scenario-based

A financial services company is building a model to predict loan defaults. The dataset is large and contains a mix of numerical and categorical features. The company wants the highest possible predictive accuracy. They are considering using either Random Forest or Gradient Boosting (e.g., XGBoost). Compare and contrast these two methods and justify which one you might choose for this task.

Solution

Feature	Random Forest (Bagging)	Gradient Boosting (Boosting)
Goal	Reduce Variance.	Reduce Bias.
Learners	Independent, deep trees (High Variance).	Sequential, shallow trees (High Bias).
Performance	Robust, rarely overfits, good baseline.	Usually higher accuracy ceiling.
Tuning	Few hyperparameters (easier).	Many hyperparameters (harder).

Recommendation: **Gradient Boosting (XGBoost/LightGBM)** is preferred for maximum accuracy. It excels with tabular data and mixed features. While harder to tune, its ability to correct errors sequentially allows it to fit complex patterns better than Random Forest.

Question 8.4: Scenario-based

You have trained an AdaBoost classifier that is performing poorly on the test set, showing signs of high bias. The weak learners are decision stumps (trees of depth 1). What specific change would you make to the AdaBoost algorithm's configuration to address this problem, and why would it help?

Solution

Problem: High Bias (Underfitting). The ensemble is too simple.

Solution: Increase the depth of the base learners (e.g., from depth 1 to depth 2 or 3).

Why? AdaBoost reduces bias by combining weak learners. If the weak learners are *too* weak (stumps only split once), they might fail to capture even basic interactions.

Increasing their complexity slightly (lower bias, higher variance) allows the boosting process to build a much more expressive final model.

Question 8.5: Conceptual

Explain the key differences between Bagging (like in Random Forest) and Boosting (like in AdaBoost) in terms of how they handle the bias-variance tradeoff.

Solution

- **Bagging:** Reduces **Variance**. Averages many high-variance (overfitted) models. The error cancels out. Formula: $Var(\text{Mean}) \approx \frac{1}{N} Var(X)$.
- **Boosting:** Reduces **Bias**. Combines many high-bias (underfitted) models. By sequentially fixing errors, the ensemble becomes complex enough to fit the data.

Question 8.6: Conceptual

What is Out-of-Bag (OOB) Error in Random Forest and how is it used?

Solution

Definition: OOB Error is the average prediction error on each training sample x_i , using only the trees that did *not* have x_i in their bootstrap sample.

Use: It acts as a free, unbiased estimate of the **generalization error** (similar to Cross-Validation) without needing a separate validation set.

Question 8.7: Conceptual

In Gradient Boosting, what is the role of the 'Learning Rate' (shrinkage)?

Solution

Role: It scales the contribution of each new tree added to the ensemble:

$$F_{new}(x) = F_{old}(x) + \eta \cdot h(x)$$

A small learning rate ($\eta < 0.1$) prevents the model from changing too rapidly, reducing overfitting and usually improving accuracy (at the cost of needing more trees).

Question 8.8: Conceptual

Explain the concept of 'Stacking' (Stacked Generalization). How does it differ from Voting?

Solution

Voting: Simple average or majority vote of base models. No learning involved in the combination.

Stacking: Uses a **Meta-Model** (Level-2 learner) to learn how to combine the predictions of base models (Level-1 learners). The meta-model learns which base models are reliable for which types of inputs.

Question 8.9: Conceptual

Which algorithm is easier to parallelize: Random Forest or Gradient Boosting? Why?

Solution

Random Forest. Each tree is built independently on a bootstrap sample. They can all be trained simultaneously on different cores/machines. Gradient Boosting is sequential (Tree t depends on Tree $t - 1$'s residuals), so it cannot be easily parallelized at the tree level.

Question 8.10: Conceptual

If your Random Forest model is overfitting, what hyperparameters should you tune? (Name two).

Solution

1. **Max Depth:** Reduce it. Shallower trees capture less noise.
2. **Min Samples per Leaf:** Increase it. Forces leaves to have more data, smoothing the predictions.
3. **Max Features:** Reduce it. Decorrelates trees further.

9 Unsupervised Learning

Question 9.1: Numerical

Given the 2D data points $(2, 2)$, $(3, 3)$, $(8, 8)$, $(9, 9)$ and initial cluster centroids $C1 = (2, 2)$ and $C2 = (3, 3)$. Perform one full iteration of the K-means algorithm. What are the new centroids?

Solution

Step 1: Assignment Step Calculate distances from each point to each centroid.

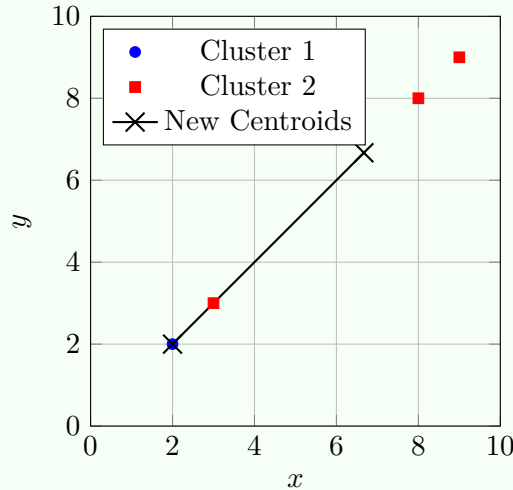
- **P1 $(2, 2)$:** $d(P1, C1) = 0$. $d(P1, C2) = \sqrt{(2-3)^2 + (2-3)^2} \approx 1.41$. $\implies$ Assign to **C1**.
- **P2 $(3, 3)$:** $d(P2, C1) \approx 1.41$. $d(P2, C2) = 0$. $\implies$ Assign to **C2**.
- **P3 $(8, 8)$:** $d(P3, C1) \approx 8.49$. $d(P3, C2) = \sqrt{(8-3)^2 + (8-3)^2} = \sqrt{25+25} \approx$

7.07. $\Rightarrow$ Assign to **C2**.

- **P4 (9, 9):** $d(P4, C1) \approx 9.90$. $d(P4, C2) = \sqrt{(9-3)^2 + (9-3)^2} \approx 8.49$. $\Rightarrow$ Assign to **C2**.

Step 2: Update Centroids Calculate the new mean of assigned points.

- **Cluster 1:** Only P1 (2, 2). New C1 = (**2, 2**).
- **Cluster 2:** P2 (3, 3), P3 (8, 8), P4 (9, 9). $x_{new} = \frac{3+8+9}{3} = \frac{20}{3} \approx 6.67$. $y_{new} = \frac{3+8+9}{3} \approx 6.67$. New C2 = (**6.67, 6.67**).



Question 9.2: Numerical

Consider a Gaussian Mixture Model (GMM) with two components, A and B. For a data point $x = 6$, the model parameters are:

- Component A: $\pi_A = 0.5, \mu_A = 5, \sigma_A = 1$
- Component B: $\pi_B = 0.5, \mu_B = 8, \sigma_B = 2$

Calculate the responsibilities (soft assignments) of component A and B for the data point $x = 6$. Use the Gaussian probability density function: $N(x|\mu, \sigma) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$.

Solution

Step 1: Calculate Unnormalized Likelihoods Ignore constants ($\frac{1}{\sqrt{2\pi}}$) as they cancel out. Include $\frac{1}{\sigma}$.

- **Likelihood A:** $\frac{1}{1} \exp\left(-\frac{(6-5)^2}{2(1)^2}\right) = e^{-0.5} \approx 0.6065$.
- **Likelihood B:** $\frac{1}{2} \exp\left(-\frac{(6-8)^2}{2(2)^2}\right) = 0.5 \cdot e^{-0.5} \approx 0.3033$.

Step 2: Weight by Priors (π)

- $P(A, x) = 0.5 \times 0.6065 \approx 0.3033$.
- $P(B, x) = 0.5 \times 0.3033 \approx 0.1516$.

Step 3: Normalize (Responsibilities) Total Evidence $Z = 0.3033 + 0.1516 = 0.4549$.

- $\gamma(A) = \frac{0.3033}{0.4549} \approx \mathbf{0.667}$.
- $\gamma(B) = \frac{0.1516}{0.4549} \approx \mathbf{0.333}$.

Note: Point 6 is closer to A's mean (5) than B's mean (8), and A has smaller variance (sharper peak), so it claims higher responsibility.

Question 9.3: Scenario-based

A marketing team wants to segment its customer base to create targeted advertising campaigns. They have data on customer demographics, purchase history, and website browsing behavior. They are considering using either K-means or a Gaussian Mixture Model (GMM). Which would you recommend and why?

Solution

Recommendation: Gaussian Mixture Model (GMM).

Why?

1. **Soft Clustering:** Customers rarely fit into just one rigid box. GMM gives probabilities (e.g., 80% "Budget", 20% "Luxury"). This nuance aids better targeting.
2. **Cluster Shape:** K-means assumes spherical clusters of equal size. Customer segments are often elliptical (correlated features, e.g., Income vs Spend) and of varying sizes. GMM handles this flexibility.

Question 9.4: Scenario-based

You are tasked with detecting fraudulent transactions from a stream of credit card data. The vast majority of transactions are legitimate. How can you use an unsupervised learning approach like Gaussian Mixture Models (GMM) for this anomaly detection task?

Solution

Approach: Density Estimation

1. **Train on Normal Data:** Fit a GMM to the available data (mostly legitimate). It will learn the high-density regions of "normal" behavior.
2. **Score New Transactions:** For each new transaction x , calculate its probability density $p(x)$ under the model.
3. **Thresholding:** If $p(x) < \epsilon$ (threshold), flag it as anomalous/fraud. Fraud typically exhibits unusual patterns (low probability under the normal distribution).

Question 9.5: Conceptual

Explain the difference between the K-means and K-medoids clustering algorithms. In what kind of situation would you prefer to use K-medoids over K-means?

Solution

Difference:

- **K-means:** Centroid is the **mean** of points (artificial point). Uses Squared Euclidean distance. Sensitive to outliers.
- **K-medoids:** Centroid is an **actual data point** (medoid). Minimizes sum of dissimilarities (any metric). Robust to outliers.

Preference: Use **K-medoids** when:

1. The data contains significant **outliers** (mean is pulled by outliers).

2. You need to use a non-Euclidean distance metric (e.g., Manhattan, Cosine) directly.

Question 9.6: Conceptual

What is the Silhouette Coefficient? What range does it fall in, and what does a value of +1 indicate?

Solution

Definition: Measures how similar a point is to its own cluster (cohesion) compared to other clusters (separation).

$$s = \frac{b - a}{\max(a, b)}$$

Range: $[-1, +1]$. **+1 Indicates:** The sample is far away from neighboring clusters and very close to its own cluster center. (Perfect clustering).

Question 9.7: Conceptual

Explain the difference between Agglomerative and Divisive Hierarchical Clustering.

Solution

- **Agglomerative (Bottom-Up):** Start with N clusters (1 point each). Iteratively merge the two closest clusters until 1 cluster remains.
- **Divisive (Top-Down):** Start with 1 cluster (all points). Iteratively split the cluster recursively until N clusters remain.

Question 9.8: Conceptual

What are the two key hyperparameters of DBSCAN? How does setting them too high or too low affect the result?

Solution

1. **Epsilon (ϵ):** Radius of neighborhood.
 - Too Small: Many points become noise (sparse).
 - Too Large: All points merge into one cluster.
2. **MinPts:** Min points to form a dense region.
3. Too Small: Noise gets clustered; many small clusters.
4. Too Large: Hard to form clusters; many points become noise.

Question 9.9: Conceptual

Why does K-Means fail to cluster non-convex shapes (like concentric circles/donuts)?

Solution

K-Means uses Euclidean distance and minimizes variance, assuming clusters are convex (spherical/blob-like). It essentially creates linear Voronoi boundaries. It cannot learn "surrounding" geometries like a ring around a core.

Question 9.10: Conceptual

In Hierarchical Clustering, what is 'Single Linkage' vs 'Complete Linkage'? Which one is prone to the 'chaining effect'?

Solution

- **Single Linkage:** Distance between closest points of two clusters. Prone to **Chaining Effect** (long, thin straggly clusters).
- **Complete Linkage:** Distance between farthest points. Tends to produce compact, spherical clusters.

10 Model Evaluation

Question 10.1: Numerical

A binary classifier for disease detection was tested on 200 people. The results are summarized in the confusion matrix below:

		Predicted Class	
		Positive	Negative
Actual Class	Positive	TP = 20	FN = 10
	Negative	FP = 5	TN = 165

Calculate the following metrics: (a) Accuracy, (b) Precision, (c) Recall (Sensitivity), and (d) F1-Score.

Solution

(a) **Accuracy** Proportion of correct predictions.

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} = \frac{20 + 165}{200} = \frac{185}{200} = \mathbf{0.925} \quad (92.5\%)$$

(b) **Precision** Proportion of positive predictions that are correct.

$$\text{Precision} = \frac{TP}{TP + FP} = \frac{20}{20 + 5} = \frac{20}{25} = \mathbf{0.80}$$

(c) **Recall (Sensitivity)** Proportion of actual positives correctly identified.

$$\text{Recall} = \frac{TP}{TP + FN} = \frac{20}{20 + 10} = \frac{20}{30} \approx \mathbf{0.667}$$

(d) **F1-Score** Harmonic mean of Precision and Recall.

$$F1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} = 2 \times \frac{0.8 \times 0.667}{0.8 + 0.667} \approx \mathbf{0.727}$$

		Predicted	
		Pos	Neg
Actual	Pos	TP (20)	FN (10)
	Neg	FP (5)	TN (165)

Question 10.2: Numerical

A logistic regression model outputs probabilities for three instances belonging to the positive class (label=1). Calculate the log-loss for these predictions. The formula for log-loss for a single instance is: $-(y \log(p) + (1 - y) \log(1 - p))$.

Instance	Predicted Probability (p)	Actual Label (y)
1	0.8	1
2	0.3	0
3	0.6	1

Solution

Step 1: Calculate Individual Losses Using natural logarithm (ln).

- **Inst 1** ($y = 1, p = 0.8$): $-\ln(0.8) \approx 0.223$.
- **Inst 2** ($y = 0, p = 0.3$): $-\ln(1 - 0.3) = -\ln(0.7) \approx 0.357$.
- **Inst 3** ($y = 1, p = 0.6$): $-\ln(0.6) \approx 0.511$.

Step 2: Average Loss

$$\text{Log Loss} = \frac{0.223 + 0.357 + 0.511}{3} = \frac{1.091}{3} \approx \mathbf{0.364}$$

Question 10.3: Scenario-based

A team develops a model to detect credit card fraud. The dataset is highly imbalanced, with only 0.1% of transactions being fraudulent. After training, the model achieves 99.9% accuracy on the test set. Why is accuracy a misleading metric here? What alternative metrics should the team focus on, and why?

Solution

Accuracy Paradox: A naive model that simply predicts "Not Fraud" for every transaction achieves 99.9% accuracy (since 99.9% are legitimate) but has 0 recall (catches no fraud). High accuracy masks poor performance on the minority class.

Better Metrics:

- **Recall (Sensitivity):** Crucial to catch all fraud cases.
- **Precision:** Important to minimize false alarms (blocking valid cards).
- **F1-Score / AUC-PR:** Balances both, focusing on the positive class.

Question 10.4: Scenario-based

Explain the bias-variance tradeoff in the context of tuning the `max_depth` parameter of a Decision Tree. What would you expect to happen to the training error and validation error as you increase `max_depth` from a small value to a very large value?

Solution

Small `max_depth`: High Bias, Low Variance. Simple model. Both Training and Validation errors are high (Underfitting).

Large `max_depth`: Low Bias, High Variance. Complex model. Training error drops to near zero (memorization). Validation error decreases initially, then rises as the model overfits noise.

Optimal Depth: The "sweet spot" (minimum validation error) before overfitting begins.

Question 10.5: Conceptual

What is K-fold cross-validation, and why is it generally preferred over a single train-validation split for model evaluation and hyperparameter tuning?

Solution

Process: Split data into K folds. Train on $K - 1$, validate on the remaining 1. Repeat K times and average the scores.

Benefits:

1. **Robust Estimate:** Reduces variance in performance estimation caused by a single "lucky" or "unlucky" random split.
2. **Data Efficiency:** All data is used for both training and validation (in different folds), which is critical for small datasets.

Question 10.6: Conceptual

Calculate the Macro-F1 and Micro-F1 scores for a 3-class problem. Class A: TP=10, FP=2, FN=5. Class B: TP=20, FP=5, FN=10. Class C: TP=50, FP=10, FN=10.

Solution

Class-wise Metrics:

- **Class A:** $P = \frac{10}{12} = 0.83$, $R = \frac{10}{15} = 0.67$, $F1 \approx 0.74$.
- **Class B:** $P = \frac{20}{25} = 0.80$, $R = \frac{20}{30} = 0.67$, $F1 \approx 0.73$.
- **Class C:** $P = \frac{50}{60} = 0.83$, $R = \frac{50}{60} = 0.83$, $F1 \approx 0.83$.

Macro-F1 (Average of F1s):

$$\frac{0.74 + 0.73 + 0.83}{3} \approx \mathbf{0.77}$$

Micro-F1 (Aggregate Counts): Total TP = 80. Total FP = 17. Total FN = 25.

$$P_{micro} = \frac{80}{97} \approx 0.825, \quad R_{micro} = \frac{80}{105} \approx 0.762$$

$$F1_{micro} = 2 \frac{0.825 \times 0.762}{0.825 + 0.762} \approx \mathbf{0.79}$$

Question 10.7: Conceptual

What is the ROC AUC (Area Under Curve)? If a model has $AUC = 0.5$, what does it mean?

Solution

Definition: Probability that the model ranks a random positive example higher than a random negative example. Ranges from 0 to 1.

AUC = 0.5: The model has no discrimination capacity. It performs equivalent to **random guessing** (flipping a coin).

Question 10.8: Conceptual

Explain the Kappa Statistic (Cohen's Kappa). When is it useful?

Solution

Definition: Measures inter-rater agreement (or classifier accuracy) corrected for chance agreement.

$$\kappa = \frac{p_o - p_e}{1 - p_e}$$

where p_o is observed agreement and p_e is expected chance agreement.

Use Case: Highly imbalanced datasets. It reveals if a high accuracy is merely due to the dominance of the majority class.

Question 10.9: Conceptual

In a medical test for a deadly disease, is High Precision or High Recall more important? Why?

Solution

High Recall is more important. Missed cases (False Negatives) are fatal. False Positives (Low Precision) are inconvenient (unnecessary further testing) but acceptable to save lives.

Question 10.10: Conceptual

What is 'Stratified' K-Fold Cross Validation? When should you use it?

Solution

Stratified K-Fold: Ensures each fold has the same class distribution as the original dataset.

When to use: Mandatory for **imbalanced datasets**. Prevents scenarios where a fold ends up with zero positive examples, which would break training or evaluation.

11 Emerging Requirements in ML

Question 11.1: Scenario-based

A bank uses a machine learning model to approve or deny loan applications. After deployment, they find that the model has a significantly lower approval rate for applicants from a protected minority group compared to the majority group, even when controlling for financial factors. What is this problem called? Describe two potential sources of this issue and one way to mitigate it.

Solution

Problem: Algorithmic Bias (Fairness Disparity).

Potential Sources:

1. **Historical Bias:** Training data reflects past human prejudices (e.g., discriminatory lending practices). The model learns to perpetuate these patterns.
2. **Proxy Variables:** Even if "Race" is removed, features like "ZIP Code" or "School" might correlate strongly with race, allowing the model to discriminate indirectly.

Mitigation: Pre-processing: Reweighting the training data. Assign higher weights to minority examples to balance their influence on the loss function, ensuring the model treats groups more equitably.

Question 11.2: Scenario-based

You have developed a highly accurate but complex deep learning model to help doctors diagnose pneumonia from chest X-rays. A senior doctor is hesitant to use it, saying, "I can't trust a black box with a patient's health. I need to know WHY it thinks this is pneumonia." How would you address the doctor's valid concern? Name and briefly describe two techniques that could provide an explanation for a single prediction.

Solution

Address Concern: Acknowledge the need for transparency. Explain that while the model is complex ("Black Box"), we can use **Explainability (XAI)** tools to visualize its decision process, building trust.

Techniques:

1. **LIME (Local Interpretable Model-agnostic Explanations):** Approximates the complex model locally with a simple one (like linear regression). It highlights regions of the X-ray (superpixels) that most contributed to the "Pneumonia" prediction.

2. **SHAP (SHapley Additive exPlanations):** Uses game theory to calculate the marginal contribution of each pixel to the prediction. It produces a heatmap showing which areas pushed the probability up (red) or down (blue).

Question 11.3: Scenario-based

A self-driving car's object detection model works perfectly in sunny weather but frequently fails to detect pedestrians at dusk or in heavy rain. This is an issue of model robustness. Describe what robustness means in this context and suggest two distinct strategies to improve it.

Solution

Robustness: The ability of a model to maintain performance despite perturbations or changes in the input distribution (e.g., lighting, weather noise).

Strategies:

1. **Data Augmentation:** Train on synthetic variations of images (darkened, blurred, added noise) to simulate adverse conditions.
2. **Adversarial Training:** Generate "worst-case" examples that fool the model and include them in training, forcing the model to learn more robust features.

Question 11.4: Scenario-based

A startup is building a recommendation engine for a new social media app. They anticipate scaling from thousands to tens of millions of users within a year. Why would a standard k-Nearest Neighbors (k-NN) approach for finding similar users be a poor choice for this application from a scalability and efficiency perspective? Suggest a more scalable alternative.

Solution

Standard k-NN Issues:

- **Prediction Latency:** Query time is $O(N)$. With millions of users, calculating distance to everyone for every recommendation is too slow for real-time use.
- **Storage:** Requires storing the entire dataset in memory.

Scalable Alternative: Approximate Nearest Neighbor (ANN). Algorithms like **Locality-Sensitive Hashing (LSH)** or HNSW bucket similar users together.

- **Result:** Search time becomes sub-linear (e.g., $O(\log N)$) by checking only relevant buckets, sacrificing negligible accuracy for massive speed.

Question 11.5: Conceptual

Differentiate between the concepts of interpretability and explainability in the context of machine learning models. Can a model be one without being the other?

Solution

- **Interpretability (White Box):** You understand the model's internal mechanics directly (e.g., Linear Regression coefficients).
- **Explainability (Post-hoc):** You use external tools to explain the predictions of a complex model (Black Box) without understanding its internal mechanics.

Relationship: Yes. A deep neural network is **explainable** (via SHAP) but not **interpretable**. A linear regression is inherently **interpretable** (the model itself is the explanation).

Question 11.6: Conceptual

What is Differential Privacy in the context of Machine Learning?

Solution

Definition: A mathematical guarantee that the output of a model (or query) does not reveal whether any specific individual's data was included in the training set.

Mechanism: Usually achieved by adding carefully calibrated noise (e.g., Laplacian noise) to the data or gradients during training.

Question 11.7: Conceptual

Explain the concept of 'Model Drift' (or Concept Drift). How can you detect it?

Solution

Definition: The statistical properties of the target variable or input features change over time, causing the model's performance to degrade (e.g., pre-COVID vs. post-COVID shopping behavior).

Detection: Monitor performance metrics (Accuracy, F1) on a sliding window of recent data. Significant drops trigger retraining.

Question 11.8: Conceptual

What is Federated Learning?

Solution

Definition: A decentralized training approach. Instead of sending user data to a central server, the model is sent to user devices (e.g., phones). Training happens locally on private data, and only the **model updates (gradients)** are sent back to the server to be aggregated.

Question 11.9: Conceptual

Differentiate between LIME and SHAP as explainability methods.

Solution

- **LIME: Local.** Perturbs input around a specific instance to fit a simple linear model. Fast but potentially unstable.
- **SHAP: Global & Local.** Based on Game Theory (Shapley values). Fairly distributes the "payout" (prediction) among features. Consistent and theoretically grounded but computationally expensive.

Question 11.10: Conceptual

What is Data Poisoning in ML security?

Solution

Definition: An adversarial attack where malicious data is injected into the training set (e.g., mislabeled images) to corrupt the model's learning process, often to create specific vulnerabilities or "backdoors" (e.g., misclassifying a stop sign with a sticky note).